

MINI LANGUAGE PROJECT DOCUMENTATION

Project: PPL-mini-lang-proj

Documentation scope: Interpreter architecture and Windows command-line integration

Source reviewed: lexer.py, parser.py, interpreter.py, main.py, README.md, Windows launcher/build scripts, mini.spec, demo files, and temp.py

CONTENTS

Part 1 - Language Interpreter Logic

1. Architecture at a glance
2. Source-to-execution pipeline
3. lexer.py: lexical analysis
4. parser.py: syntax analysis and AST
5. interpreter.py: evaluation and runtime state
6. main.py: orchestration and entry point
7. Supported language behavior
8. Error and scope behavior

Part 2 - Environment & CLI Integration

9. Runtime prerequisites and project layout
10. Windows launchers
11. CLI modes and commands
12. PowerShell and cmd environment setup
13. Windows packaging and PyInstaller configuration
14. Demonstration files and development notes
15. End-to-end operational flow

PART 1: LANGUAGE INTERPRETER LOGIC

1. ARCHITECTURE AT A GLANCE

Mini is a small tree-walk interpreter implemented in Python. Its active execution engine is divided into four cooperating modules:

- lexer.py converts source text into tokens.
- parser.py converts tokens into an abstract syntax tree (AST) represented by dataclass nodes.
- interpreter.py evaluates AST nodes using environments and runtime objects.
- main.py provides the public command-line interface, file execution, inline-code execution, and the REPL.

The system is intentionally direct: there is no bytecode compiler or intermediate VM. Each parsed node is visited immediately by `Interpreter.visit()`.

2. SOURCE-TO-EXECUTION PIPELINE

For a source file or code string, execution follows this sequence:

1. main.py receives command-line arguments or a REPL input line/block.
2. Lexer normalizes Windows line endings and scans characters.
3. `Lexer.tokenize()` emits tokens and appends EOF.
4. Parser consumes the token list and builds statement and expression nodes.
5. `Interpreter.run()` visits the top-level node list in order.
6. Each node evaluates against an Environment containing variables and parent-scope links.
7. print statements write values to standard output. Errors are reported by main.py.

Conceptual pipeline:

source text -> Lexer -> tokens -> Parser -> AST nodes -> Interpreter -> output/state

3. lexer.py: LEXICAL ANALYSIS

Purpose

lexer.py defines the vocabulary of the language and performs character-level scanning. It is responsible for recognizing valid lexical units, not for understanding statement structure.

TokenType

TokenType enumerates:

- Literals and names: NUMBER, STRING, IDENTIFIER, KEYWORD.
- Arithmetic: PLUS, MINUS, MUL, DIV.
- Assignment and comparisons: EQUALS, DOUBLE_EQUALS, NOT_EQUALS, GREATER, LESS, GREATER_EQUALS, LESS_EQUALS.
- Delimiters: LPAREN, RPAREN, COMMA, COLON.
- Structure markers: NEWLINE and EOF.

Token

Token stores a token type and an optional value. Examples include NUMBER:5, STRING:hello, IDENTIFIER:x, and KEYWORD:while. Its repr is useful during debugging.

Keywords

The reserved words are let, if, else, while, function, return, print, true, false, and end. Identifiers matching this list become KEYWORD tokens; other alphabetic names become IDENTIFIER tokens.

Scanning behavior

- Spaces and tabs are ignored.
- Newlines are preserved as NEWLINE tokens so the parser can separate statements.
- A hash character begins a comment that continues to the end of the line.
- Digit runs become integer NUMBER tokens.
- Names may contain letters, digits, and underscores after the first character.
- Double-quoted text becomes a STRING token. Unterminated strings raise an exception.
- One- and two-character operators are recognized, including ==, !=, >=, and <=.
- Parentheses, commas, and colons are emitted as delimiter tokens.
- Any unrecognized character raises an exception immediately.

The lexer normalizes CRLF and CR to LF, allowing source files created on Windows to be parsed consistently.

4. parser.py: SYNTAX ANALYSIS AND AST

Purpose

parser.py defines the AST data model and turns the token stream into structured nodes. Parsing is recursive descent with separate methods for statements, comparisons, arithmetic, terms, and factors.

AST node types

- NumberNode(value): integer literal.
- StringNode(value): string literal.
- BooleanNode(value): true or false.
- VarAccessNode(name): read an existing variable.
- VarAssignNode(name, value, declare): declaration with let when declare is true, or update when false.
- BinaryOpNode(left, operator, right): arithmetic or comparison expression.
- PrintNode(value): print a computed value.
- IfNode(condition, body, else_body): conditional branches.
- WhileNode(condition, body): repeated block.
- FunctionNode(name, parameters, body): named function definition.

- CallNode(name, arguments): named function call.
- ReturnNode(value): function return operation.

Top-level parsing

Parser.parse() skips leading and inter-statement newlines, repeatedly parses statements until EOF, and returns a list of nodes. statement() dispatches based on the current keyword or identifier. Assignments and calls are recognized using lookahead.

Blocks

if, while, and function blocks require a colon and terminate with end. An if block may contain else followed by a colon. Missing delimiters, block terminators, names, parentheses, or colons produce descriptive exceptions such as Expected 'end' or Expected ':'.

Expression precedence

The parser gives multiplication and division higher precedence than addition and subtraction. Comparisons are parsed above arithmetic. Parentheses override the normal precedence. Function calls are recognized when an identifier is followed by an opening parenthesis.

5. interpreter.py: EVALUATION AND RUNTIME STATE

Environment

Environment stores variables in a dictionary and optionally references a parent environment. get() searches the current scope and then parents. set() creates or replaces a name in the current scope. assign() updates an existing name in the nearest scope where it is found; assigning an unknown name raises Undefined variable.

This parent-link model supports lexical lookup for functions and child scopes for control-flow blocks.

Interpreter.visit()

visit() evaluates each AST node type:

- Literal nodes return their Python value.
- VarAccessNode reads from the environment.
- VarAssignNode evaluates the right side, then declares or updates the name.
- PrintNode evaluates and writes the value with Python print().
- BinaryOpNode delegates to visit_binary().
- IfNode checks for a real bool, creates a child environment for the selected branch, and evaluates its statements.
- WhileNode repeatedly evaluates a bool condition and executes its body in a new child environment per iteration.
- FunctionNode captures the current environment in a Function object and binds it by name.
- CallNode delegates to call_function().
- ReturnNode evaluates its value and raises ReturnException to unwind the function body.

Operators and type rules

- + supports two values of the same type when that type is int or str.
- -, *, and / require integers; division is integer floor division.
- Division by zero raises Cannot divide by zero.
- == requires matching types and equal values.
- != returns true for differing types or differing values.
- >, <, >=, and <= require integers.
- Booleans are explicitly rejected as numbers even though Python bool is an int subclass.

Functions and returns

A Function stores its FunctionNode and the environment active at definition time. Calling a function checks that the target is a Function and that argument count matches parameter count. Arguments are evaluated in the

caller environment. A new function environment is created with the captured environment as parent, parameters are bound, and the body runs there. ReturnException carries the return value out of nested evaluation. A function without return produces None.

run()

Interpreter.run() visits top-level nodes in source order and returns the last visit result. Statements such as print, if, while, and function definitions normally return None.

6. main.py: ORCHESTRATION AND ENTRY POINT

run_source(source, environment=None) is the reusable execution function. It performs the complete Lexer -> Parser -> Interpreter sequence and optionally reuses an Environment, which is important for the persistent REPL.

is_complete(source) counts opening block keywords if, while, and function against end keywords. It lets the REPL collect multi-line blocks before execution. The check is intentionally lightweight and is used only to decide whether more input is needed.

repl() prints the version banner, keeps one Environment for the session, supports exit and quit, accepts blank-line handling, and displays errors without terminating the session.

read_file(path) reads UTF-8 source files.

build_parser() defines the command-line interface:

- An optional script path runs a Mini source file.
- -c or --code runs an inline code string.
- --version prints the interpreter version.
- With no arguments, the REPL starts.

main() selects the requested mode, converts missing-file errors and other exceptions into stderr messages, and returns process status 0 for successful execution or 1 for failure. The module guard makes main.py directly executable by Python and suitable as the PyInstaller entry script.

7. SUPPORTED LANGUAGE BEHAVIOR

Variables:

```
let count = 5
count = count + 1
```

Values:

- Integers, double-quoted strings, and booleans true and false.

Output:

```
print("Hello " + name)
```

Blocks:

```
if count > 0:
  print(count)
else:
  print("empty")
end
```

```
while count < 10:
  count = count + 1
end
```

Functions:

```
function add(a, b):  
    return a + b  
end
```

```
print(add(5, 3))
```

Comments begin with # and continue to the end of the line. Block syntax uses a colon and closes with end. Newlines separate statements.

8. ERROR AND SCOPE BEHAVIOR

Lexical errors are raised for illegal characters and unterminated strings. Syntax errors are raised by the parser for malformed expressions, missing names, missing separators, and missing end markers. Runtime errors cover undefined names, invalid assignment, invalid operand types, invalid conditions, invalid function calls, incorrect argument counts, and division by zero.

main.py catches errors at the command boundary and prints Error: ... to stderr. The REPL catches errors per input unit and continues running. The interpreter uses child environments for if branches and while iterations, while assignment searches parents so an existing outer variable can be updated.

```
=====
PART 2: ENVIRONMENT & CLI INTEGRATION
=====
```

9. RUNTIME PREREQUISITES AND PROJECT LAYOUT

README.md states the supported baseline as Windows 10 or newer and Python 3.10 or newer. Verify Python with:

```
python --version
```

Important project files:

- main.py: CLI and interpreter entry point.
- lexer.py, parser.py, interpreter.py: active execution engine.
- mini.cmd: cmd launcher with compiled-executable fallback.
- mini.ps1: PowerShell launcher with the same fallback order.
- mini-shell.cmd and mini-shell.ps1: open a dedicated Command Prompt window.
- build_windows.ps1 and build_windows.cmd: PyInstaller build entry points.
- mini.spec: PyInstaller analysis and executable configuration.
- README.md: user-facing setup and command reference.
- demo/hello.mini: sample program covering variables, output, a loop, and a function.
- demo/recursion.mini: present but currently empty.
- temp.py: older prototype containing an earlier lexer/parser/interpreter/REPL design; it is not imported by the active main.py pipeline.
- build/ and dist/: generated packaging artifacts. The current workspace includes build output; dist may be generated or absent depending on the last build.

10. WINDOWS LAUNCHERS

mini.cmd

This batch launcher determines its own directory through %~dp0. It prefers, in order:

1. dist\mini.exe from a one-file build.
2. dist\mini\mini.exe from a one-directory build.
3. python main.py as the source fallback.

It forwards all command-line arguments with %* and returns the selected process exit code.

mini.ps1

This PowerShell launcher resolves the project root, constructs paths for both compiled layouts, checks them with Test-Path, invokes the first available executable, and otherwise invokes Python with main.py. Remaining arguments are collected in MiniArgs and forwarded as an array.

mini-shell.cmd

This wrapper starts a new Command Prompt window using cmd /k and invokes mini.cmd from the project directory. It preserves arguments and leaves the interactive window open.

mini-shell.ps1

This PowerShell wrapper builds a cmd.exe argument list and uses Start-Process to open a new Command Prompt window running mini.cmd. It accepts all remaining arguments.

11. CLI MODES AND COMMANDS

From cmd.exe in the project directory:

Start the REPL:

mini.cmd

Run a source file:

mini.cmd demo\hello.mini

Run inline code:

mini.cmd -c "print(1 + 2)"

Show version:

mini.cmd --version

Open a separate command window:

mini-shell.cmd

From PowerShell in the project directory:

Start the REPL:

.\mini.ps1

Run a source file:

.\mini.ps1 .\demo\hello.mini

Run inline code:

.\mini.ps1 -c "print(1 + 2)"

Show version:

.\mini.ps1 --version

Open a separate command window:

.\mini-shell.ps1

Direct Python invocation is also available:

python main.py demo\hello.mini

python main.py -c "print(1 + 2)"

12. POWERSHELL AND CMD ENVIRONMENT SETUP

To make mini.cmd discoverable anywhere in the current cmd.exe session, change to the project directory and prepend the current directory to PATH:

cd /d "..\PPL-mini-lang-proj"

```
set "PATH=%CD%;%PATH%"
mini demo\\hello.mini
```

In PowerShell, prepend the current location for the current session:

```
$env:Path = "$PWD;$env:Path"
mini demo\\hello.mini
```

PowerShell may block local script execution depending on policy. For the current terminal session only, use:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

The launcher scripts are designed to be run from the project checkout, but they resolve their own directory so source and compiled execution remain stable even when called from another working directory.

13. WINDOWS PACKAGING AND PYINSTALLER CONFIGURATION

build_windows.cmd is a thin cmd-compatible adapter. It invokes PowerShell with execution-policy bypass and passes through all arguments, returning the PowerShell exit code.

build_windows.ps1 performs the actual build:

1. Accepts the optional -OneFile switch.
2. Resolves and switches to the project root.
3. Tests whether PyInstaller can be imported.
4. Installs or upgrades PyInstaller with python -m pip if needed.
5. Builds main.py as a console application named mini.
6. Reports the resulting output path.

Default build command:

```
.\build_windows.ps1
```

Default output:

```
dist\\mini\\mini.exe
```

Single-file build:

```
.\build_windows.ps1 -OneFile
```

Single-file output:

```
dist\\mini.exe
```

The first build may require internet access because the script installs PyInstaller when it is unavailable.

mini.spec

The spec file describes a console executable named mini. Analysis starts from main.py, with no custom data files, binaries, hidden imports, hooks, or exclusions. The PYZ archive contains analyzed Python modules, and EXE is configured for console operation. The active imports from main.py pull in lexer.py, parser.py, and interpreter.py through normal Python module discovery.

The launcher fallback order matches both supported PyInstaller layouts, which allows the same command to work after either a default one-directory build or a -OneFile build.

14. DEMONSTRATION FILES AND DEVELOPMENT NOTES

demo/hello.mini is the principal working demonstration. It declares an integer and string, concatenates and prints a greeting, loops while x is less than 8, and defines and calls add(a, b).

demo/recursion.mini exists but is empty in the scanned workspace. It should not be presented as an executable recursion example until content is added.

temp.py is a historical or experimental all-in-one implementation. It contains earlier versions of tokenization, AST

classes, interpretation, and a REPL, but the production entry point imports the separate modules instead. Operational documentation should therefore use main.py and the split engine modules as the source of truth.

15. END-TO-END OPERATIONAL FLOW

Source execution from cmd or PowerShell follows this environment path:

user command -> mini.cmd or mini.ps1 -> compiled mini.exe if present, otherwise python main.py -> Lexer -> Parser -> Interpreter -> stdout/stderr and exit code

For a packaged release:

build_windows.cmd or build_windows.ps1 -> PyInstaller -> dist\mini.exe or dist\mini\mini.exe -> launcher detection -> packaged main entry point

For interactive use:

launcher -> main.py -> repl() -> persistent Environment -> repeated run_source() calls

For a file:

launcher -> main.py script argument -> read_file() -> run_source() -> tokenization -> parsing -> evaluation

For inline code:

launcher -> main.py -c/--code -> run_source() -> tokenization -> parsing -> evaluation

Operational checklist:

1. Confirm Python 3.10+ with `python --version`.
2. Open cmd.exe or PowerShell in the project directory.
3. Run mini.cmd, mini.ps1, or direct python main.py.
4. Test demo\hello.mini before using a custom program.
5. Build with build_windows.ps1 when a standalone executable is needed.
6. Use --version to verify which entry point responds.
7. If PowerShell refuses a local script, apply the process-scoped execution-policy command.

END OF DOCUMENTATION